

PROJECT:01

TITLE : AI BLOGNEST API - GENERATE HIGH-QUALITY BLOG CONTENT USING AI

AI BlogNest API Introduction:

In today's digital world, blogs are widely used to share knowledge, opinions, tutorials, news, and experiences. However, creating quality blog content manually requires considerable time and effort.

AI BlogNest combines a traditional blog management system with Artificial Intelligence. The system allows users to manage their blogs and provides AI-based features such as:

- AI blog generation
- Blog summarization
- User authentication
- Blog creation and editing
- Blog viewing
- Blog deletion
- Secure API access

The project uses **Node.js, Express.js, MongoDB, JWT authentication, and Google Gemini AI.**

AI BlogNest API Description:

AI BlogNest is a full-stack web application developed using the **MERN-oriented technology approach**, with Node.js and Express.js used for the backend and MongoDB used as the database.

Blog Management

Users can:

- Register an account
- Login securely
- Create blogs
- View blogs
- Update blogs
- Delete blogs

AI Features:

The application uses Gemini AI to:

- Generate complete blog content from a topic
- Summarize lengthy blog content
- Assist users in creating content quickly

The backend provides REST APIs that can be tested using **Thunder Client** or similar API testing tools.

Objectives:

The main objectives of AI BlogNest are:

1. To develop an AI-powered blogging platform.
2. To provide secure user registration and login.
3. To implement JWT-based authentication.
4. To allow users to create and manage blog posts.
5. To provide AI-based blog generation.

Problem Statement:

Traditional blogging platforms mainly depend on users to manually create and edit their content.

This creates several problems:

- Writing content takes more time.
- Users may find it difficult to generate ideas.
- Long blog posts are difficult to summarize manually.

Therefore, there is a need for a blogging platform that combines **blog management with AI-powered content assistance**.

AI BlogNest addresses these problems by integrating AI features into the blogging workflow.

Existing System

In the existing blogging system, users generally:

1. Register or log in.
2. Manually write blog content.
3. Edit the content.
4. Publish the blog.
5. Read or manage blogs.

Limitations

- No automatic content generation.
- No automatic summarization.
- More time required for content creation.
- Users need to manually generate ideas.

Proposed System

The proposed **AI BlogNest** system adds AI functionality to the traditional blog management system.

Proposed Features

- User registration
- User login
- AI blog generation
- AI blog summarization
- MongoDB database

Main Workflow

User



Registration / Login



Blog Management



AI Assistance

├— Generate Blog

└— Summarize Blog



MongoDB

Scenario-Based Case Study:

Scenario

Consider a student who wants to create a blog about Artificial Intelligence in Education.

Normally, the student needs to:

- Search for information.
- Decide the blog structure.
- Write an introduction.
- Write multiple paragraphs.
- Create a conclusion.
- Review the content.

Using AI BlogNest, the student can simply provide:

Software Requirements:

Software	Requirement
Operating System	Windows 10/11
Programming Language	JavaScript
Runtime	Node.js
Backend	Express.js
Database	MongoDB
AI Service	Google Gemini API
Authentication	JWT

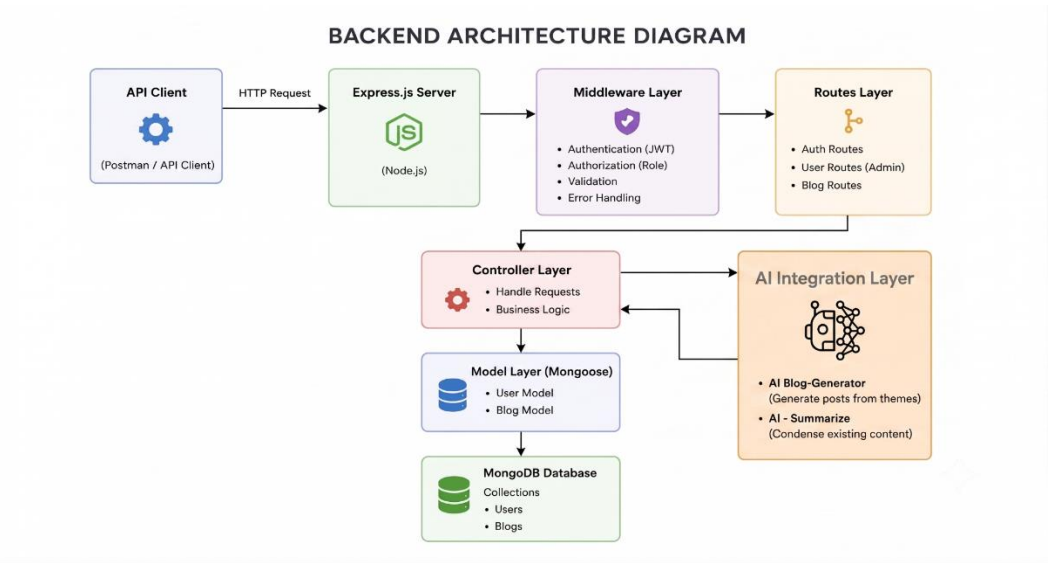
Hardware Requirements:

Minimum hardware requirements:

Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	1 GB free space
Internet	Required for Gemini API
Display	Standard monitor
Keyboard	Required
Mouse	Required

Project Architecture:

AI BlogNest follows a layered backend architecture.



Architecture Components Explanation

1. Client

The client sends requests to the backend.

Examples:

- Register
- Login
- Create blog
- Generate AI blog
- Summarize blog

2. Express Server

Express handles HTTP requests and provides APIs.

3. Authentication

JWT is used to authenticate users.

4. Routes

Routes determine which operation should be performed.

5. Models

MongoDB models define the structure of user and blog data.

6. MongoDB

MongoDB stores:

- User information
- Blog information

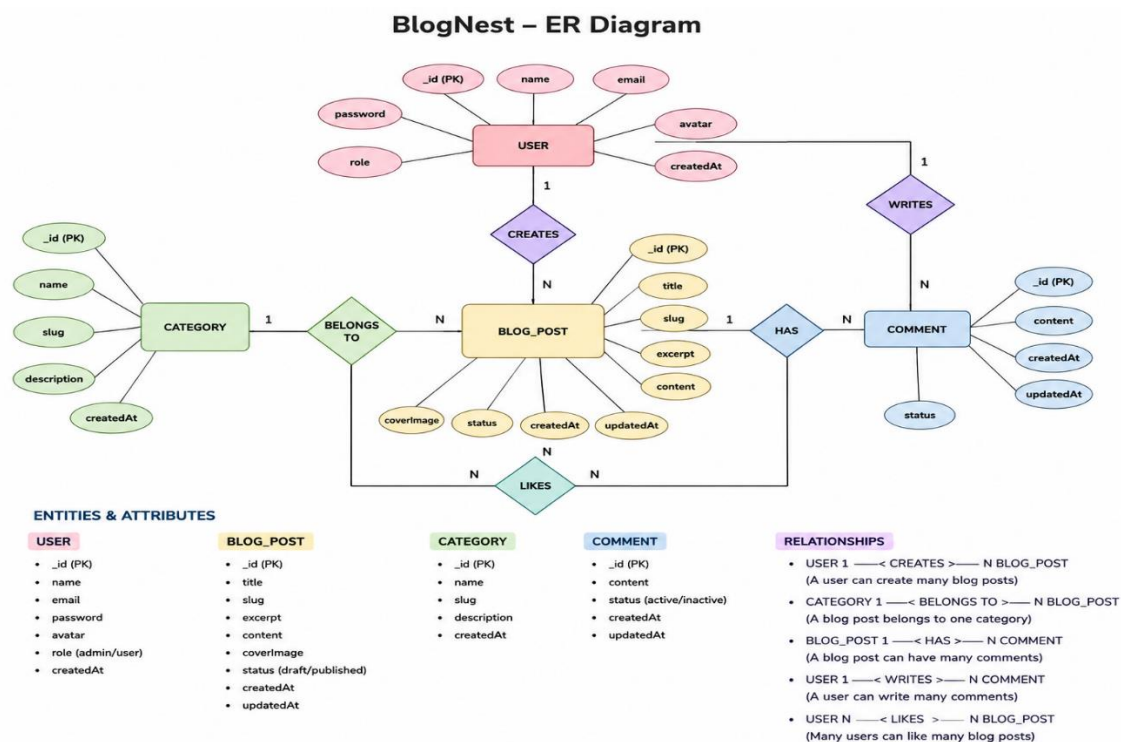
7. Gemini AI

Gemini provides AI functionality for:

- Blog generation
- Summarization

Database / ER Diagram Explanation:

AI BlogNest primarily contains two important



Relationship:

One user can create multiple blogs.

Therefore:

USER 1 : M BLOG

User Entity

The User entity stores information about registered users.

Attributes

Attribute	Description
userId	Unique user identifier
name	User name
email	User email
password	Encrypted/hashed password
createdAt	Account creation date

Blog Entity

The Blog entity stores blog information.

Attributes

Attribute	Description
blogId	Unique blog identifier
title	Blog title
content	Blog content
summary	AI-generated summary
authorId	User who created the blog
createdAt	Creation date
updatedAt	Last update date

Relationship Between Entities

The relationship between User and Blog is:

One-to-Many relationship

One User

|

| creates

|

+---- Blog 1

|

+---- Blog 2

|

+---- Blog 3

A single user can create many blogs.

A particular blog belongs to one author.

Key Features

User Features

- Registration
- Login
- Secure authentication

Blog Features

- Create blog
- Read blog

- Update blog
- Delete blog

AI Features

- Generate blog
- Summarize blog

Security Features

- Password hashing
- JWT authentication
- Protected routes

Database Features

- User storage
- Blog storage
- User-blog relationship

Roles and Responsibilities

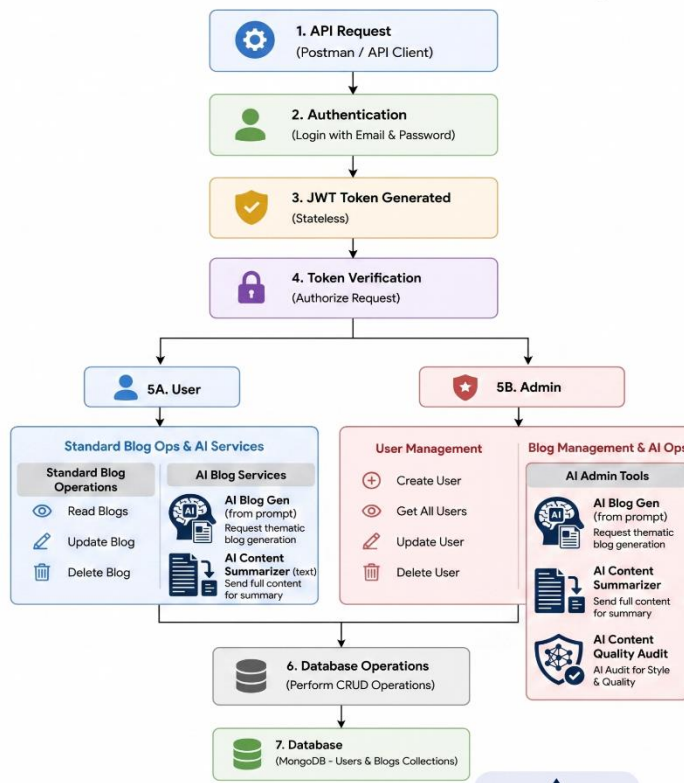
User

The user can:

- Register
- Login
- Create blogs
- View blogs
- Generate AI content
- Summarize content

User Flow:

AI-BLOG USER FLOW DIAGRAM (Backend Only)



Technology Stack

Frontend

- HTML
- CSS
- JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB
- Mongoose

Authentication

- JSON Web Token
- bcrypt

Artificial Intelligence

Google Gemini API

Testing

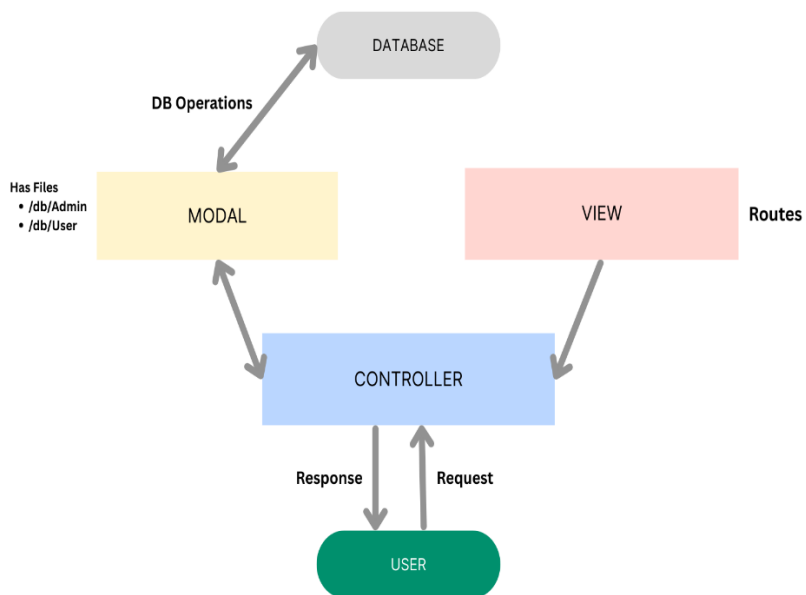
- Thunder Client

Development Tool

- Visual Studio Code

MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple component.



Project Folder Structure

A suitable project structure is:

AI-BlogNest/

|

|— controllers/

| |— authController.js

| |— blogController.js

| |— aiController.js

|

|— middleware/

| |— authMiddleware.js

|

|— models/

| |— User.js

| |— Blog.js

|

|— routes/

| |— authRoutes.js

| |— blogRoutes.js

| |— aiRoutes.js

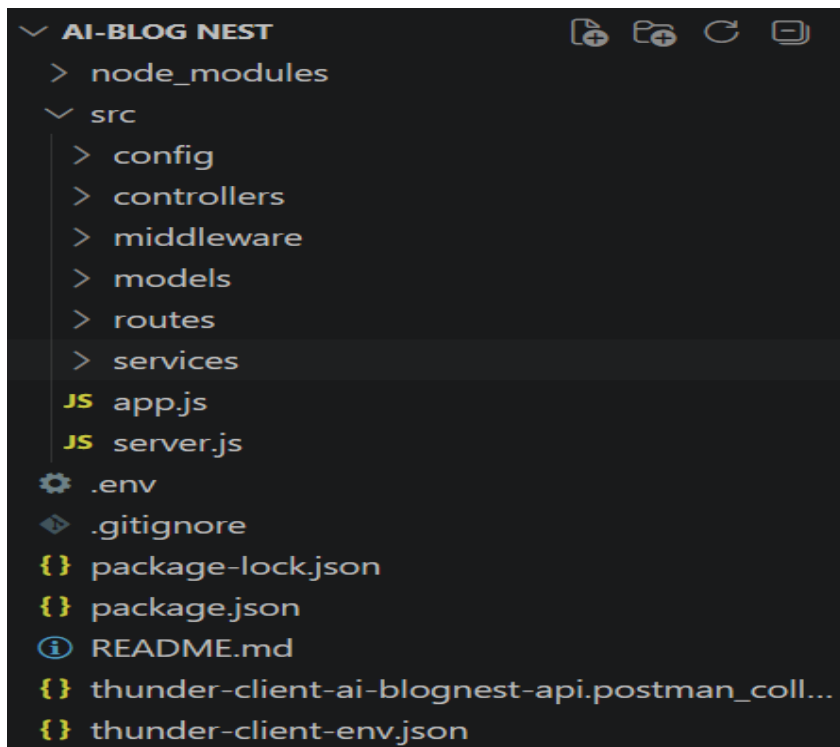
|

|— config/

```
|   └─ db.js
|
|
├─ .env
├─ .gitignore
├─ app.js
├─ server.js
├─ package.json
└─ README.md
```

Project Setup

First create the project folder:



Server setup (npm init)

```
mkdir AI-BlogNest
```

```
cd AI-BlogNest
```

Initialize Node.js:

```
npm init -y
```

```
npm init
```

The following command initializes a Node.js project:

```
npm init -y
```

```
package.json
```

The package.json file contains:

- Project name
- Version
- Scripts
- Dependencies
- Project information

Package Installation

Install the required packages:

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
```

For Gemini:

```
npm install @google/generative-ai
```

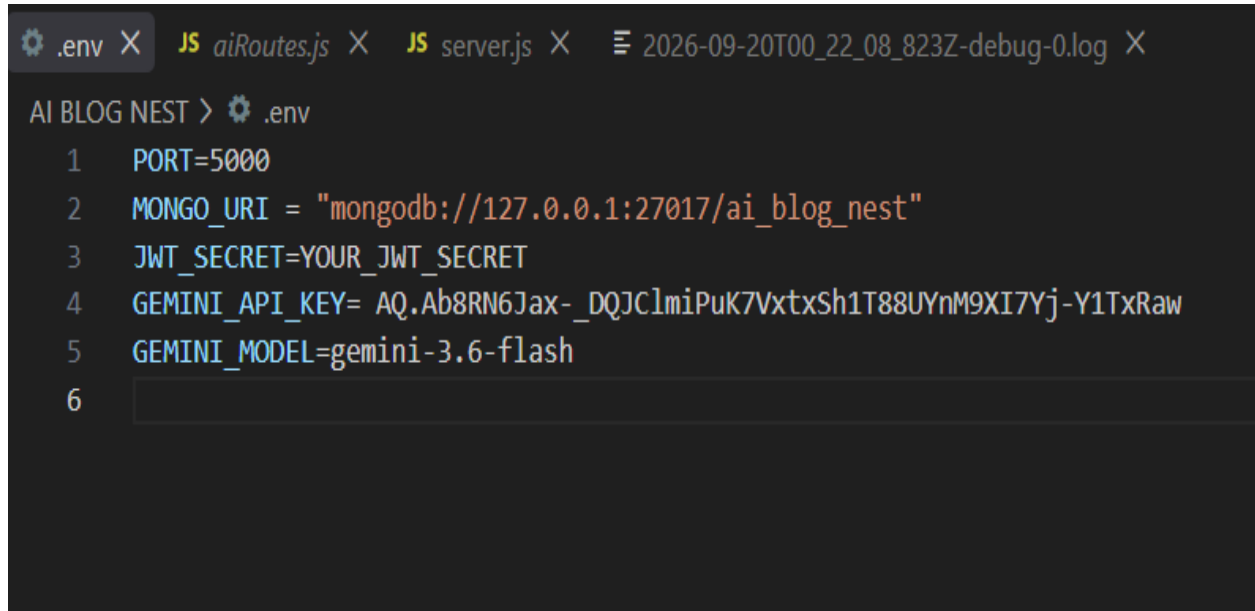
For development:

```
npm install --save-dev nodemon
```

Important Packages

Package	Purpose
express	Backend server
mongoose	MongoDB connection
dotenv	Environment variables
bcryptjs	Password hashing
jsonwebtoken	Authentication
cors	Cross-origin requests

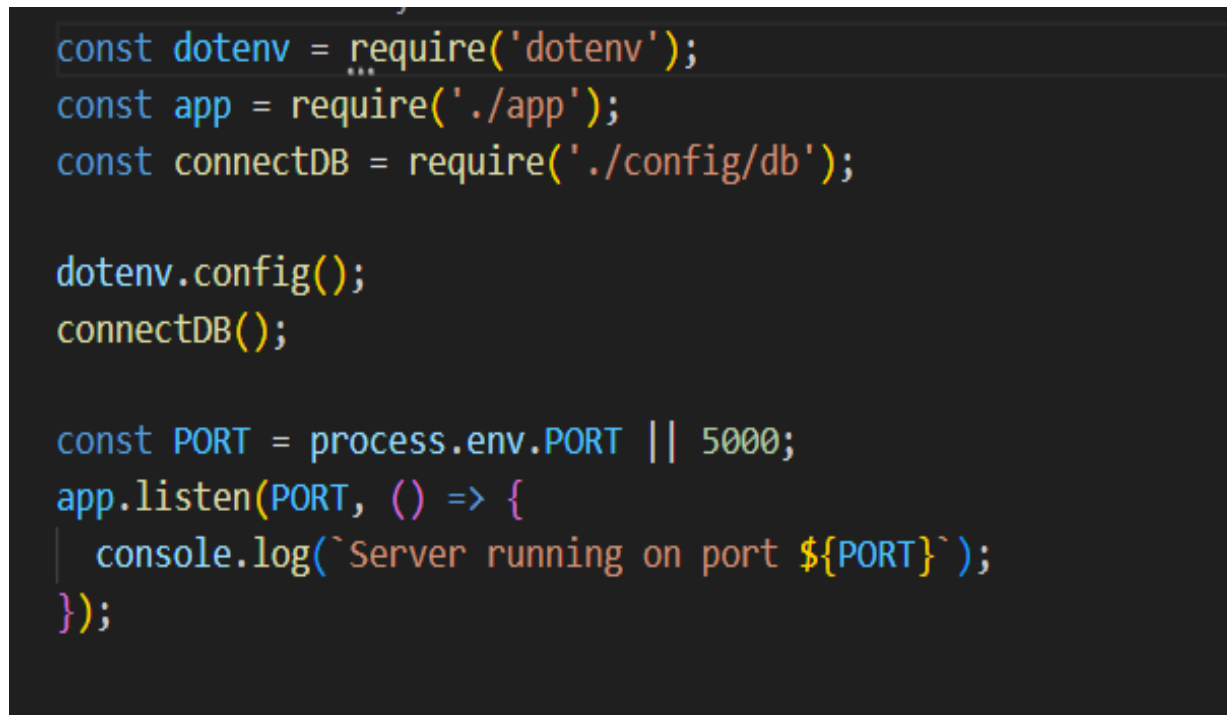
.env Configuration:



The screenshot shows a code editor with several tabs at the top: `.env`, `JS aiRoutes.js`, `JS server.js`, and a log file `2026-09-20T00_22_08_823Z-debug-0.log`. The `.env` tab is active, displaying the following configuration:

```
AI BLOG NEST > .env
1  PORT=5000
2  MONGO_URI = "mongodb://127.0.0.1:27017/ai_blog_nest"
3  JWT_SECRET=YOUR_JWT_SECRET
4  GEMINI_API_KEY= AQ.Ab8RN6Jax-_DQJClmiPuK7VxtxSh1T88UYnM9XI7Yj-Y1TxRaw
5  GEMINI_MODEL=gemini-3.6-flash
6
```

server.js Code :



```
const dotenv = require('dotenv');
const app = require('./app');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

app.js Code:

```
.env X JS server.js X JS app.js X 2026-09-20T00_22_08_823Z-debug-0.log X
AI BLOG NEST > src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22    res.json({ message: 'AI BlogNest API is running' });
23  });
```

MongoDB Connection Code

config/db.js

```
.env X JS server.js X JS db.js X 2026-09-20T00_22_08_823Z-debug-0.log X
AI BLOG NEST > src > config > JS db.js > connectDB
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
```

MongoDB Connected is displayed.

User Model Code :

models/User.js

```
.env X JS server.js X JS User.js X 2026-09-20T00_22_08_823Z-debug-0.log X
AI BLOG NEST > src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },

```

Blog Model Code

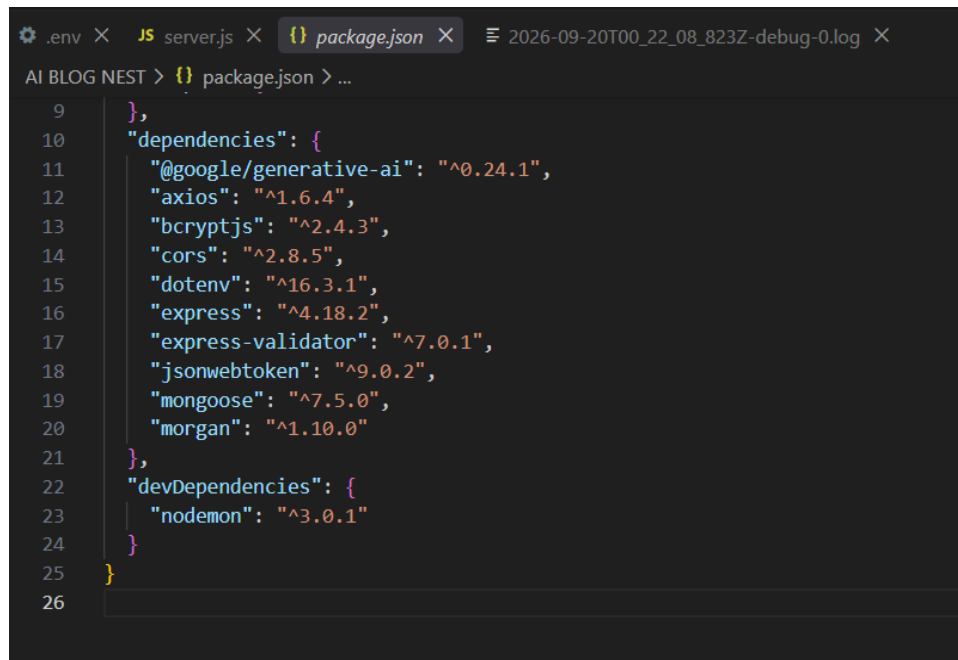
models/Blog.js

```
.env X JS server.js X JS Blog.js X 2026-09-20T00_22_08_823Z-debug-0.log X
AI BLOG NEST > src > models > JS Blog.js > ...
1  const mongoose = require('mongoose');
2
3  const blogSchema = new mongoose.Schema(
4    {
5      title: {
6        type: String,
7        required: [true, 'Title is required'],
8        trim: true,
9      },
10     content: {
11       type: String,
12       required: [true, 'Content is required'],
13     },
14     category: {
15       type: String,
16       required: [true, 'Category is required'],
17       trim: true,
18     },
19     author: {
20       type: mongoose.Schema.Types.ObjectId,
21       ref: 'User',
22       required: true,
23     },

```

JWT Code

JWT stands for JSON Web Token.



```
.env X JS server.js X {} package.json X 2026-09-20T00_22_08_823Z-debug-0.log X
AI BLOG NEST > {} package.json > ...
  9      },
 10      "dependencies": {
 11        "@google/generative-ai": "^0.24.1",
 12        "axios": "^1.6.4",
 13        "bcryptjs": "^2.4.3",
 14        "cors": "^2.8.5",
 15        "dotenv": "^16.3.1",
 16        "express": "^4.18.2",
 17        "express-validator": "^7.0.1",
 18        "jsonwebtoken": "^9.0.2",
 19        "mongoose": "^7.5.0",
 20        "morgan": "^1.10.0"
 21      },
 22      "devDependencies": {
 23        "nodemon": "^3.0.1"
 24      }
 25    }
 26  }
```

Blog CRUD Code

CRUD means:

- C – Create
- R – Read
- U – Update
- D – Delete

AI Gemini Configuration

Gemini is used to provide AI-powered functionality.

Middleware:

Authentication Middleware

```
server.js × POS localhost:5000/api/auth/r... × POS localhost:5000/api/auth/l... × POS localhost:5000/api/blogs × POS POST localhost:5000/api/a... ×
AI BLOG NEST > src > middleware > JS authMiddleware.js > ...
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const authMiddleware = async (req, res, next) => {
5    const authHeader = req.headers.authorization;
6    if (!authHeader || !authHeader.startsWith('Bearer ')) {
7      return res.status(401).json({ message: 'Authorization token missing' });
8    }
9
10   const token = authHeader.split(' ')[1];
11
12   try {
13     const decoded = jwt.verify(token, process.env.JWT_SECRET);
14     req.user = await User.findById(decoded.id).select('-password');
15     if (!req.user) {
16       return res.status(401).json({ message: 'Invalid authentication token' });
17     }
18     next();
19   } catch (error) {
20     return res.status(401).json({ message: 'Token is invalid or expired' });
21   }
22 };
23
```

The middleware:

1. Gets the JWT.
2. Verifies the JWT.
3. Identifies the user.
4. Allows access to protected routes.

Project Executions:

· Install Postman or ThunderClient (VS Code extension).

Start the backend server:

“npm start”

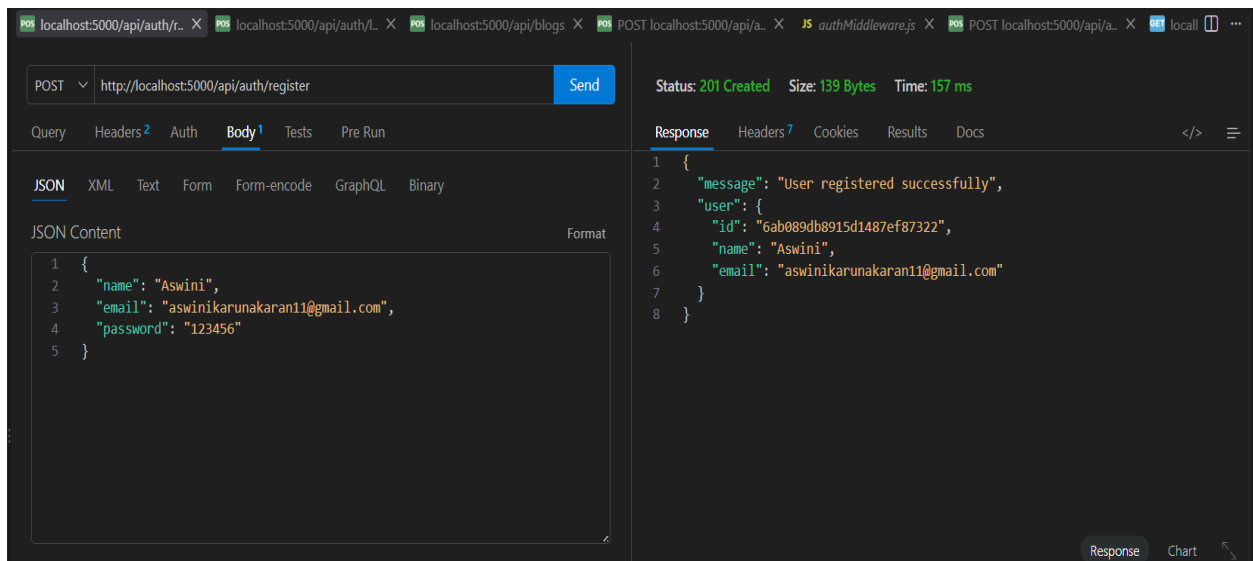
API Testing :

Routes

Authentication Routes

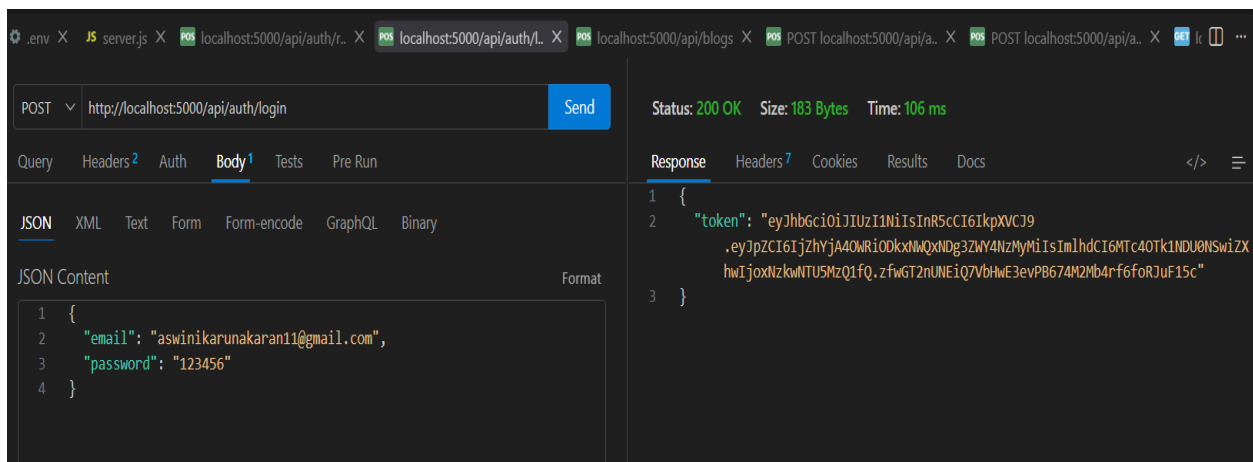
User Registration :

POST /api/auth/register

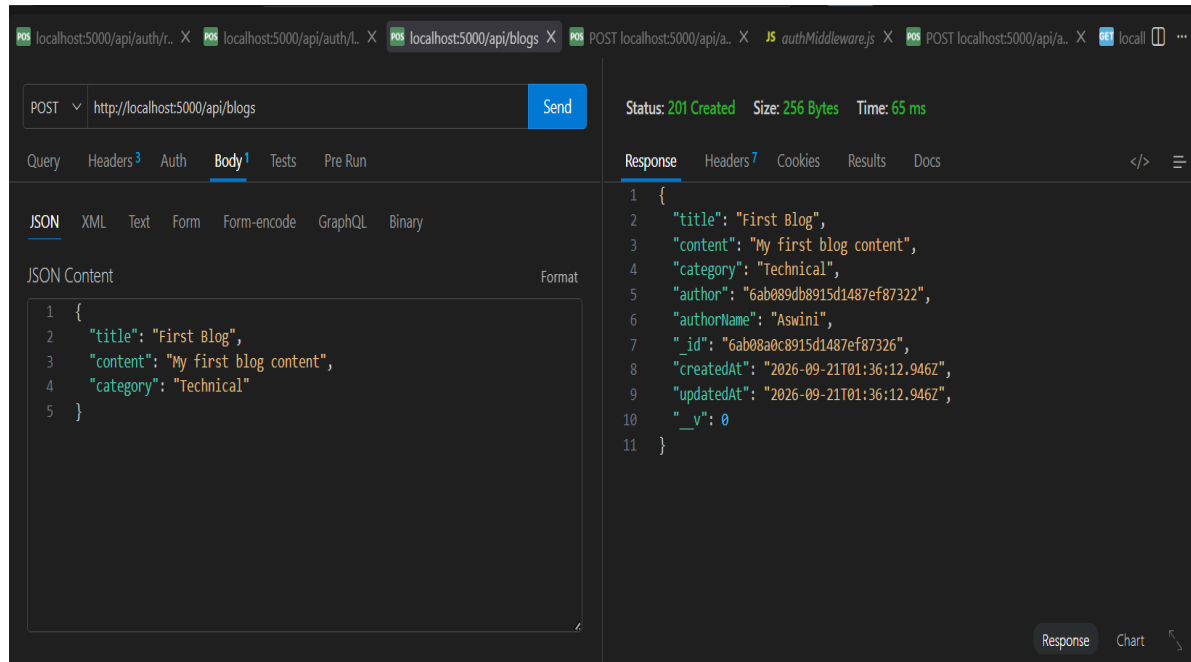


User Login:

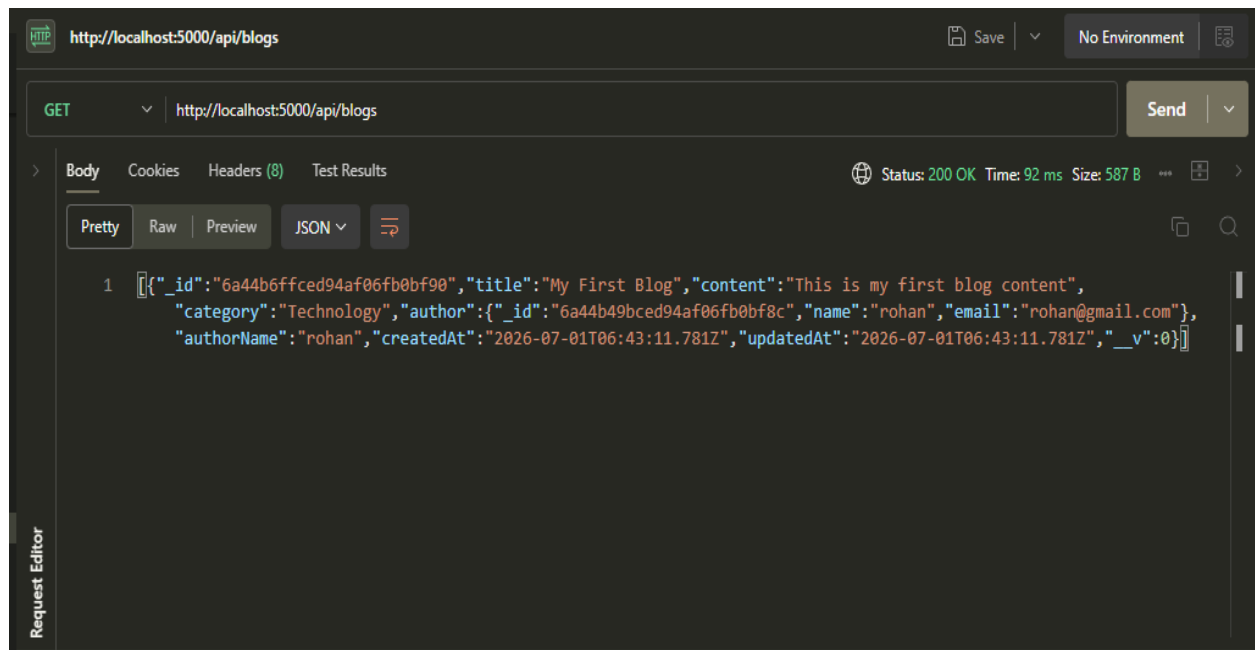
POST /api/auth/login



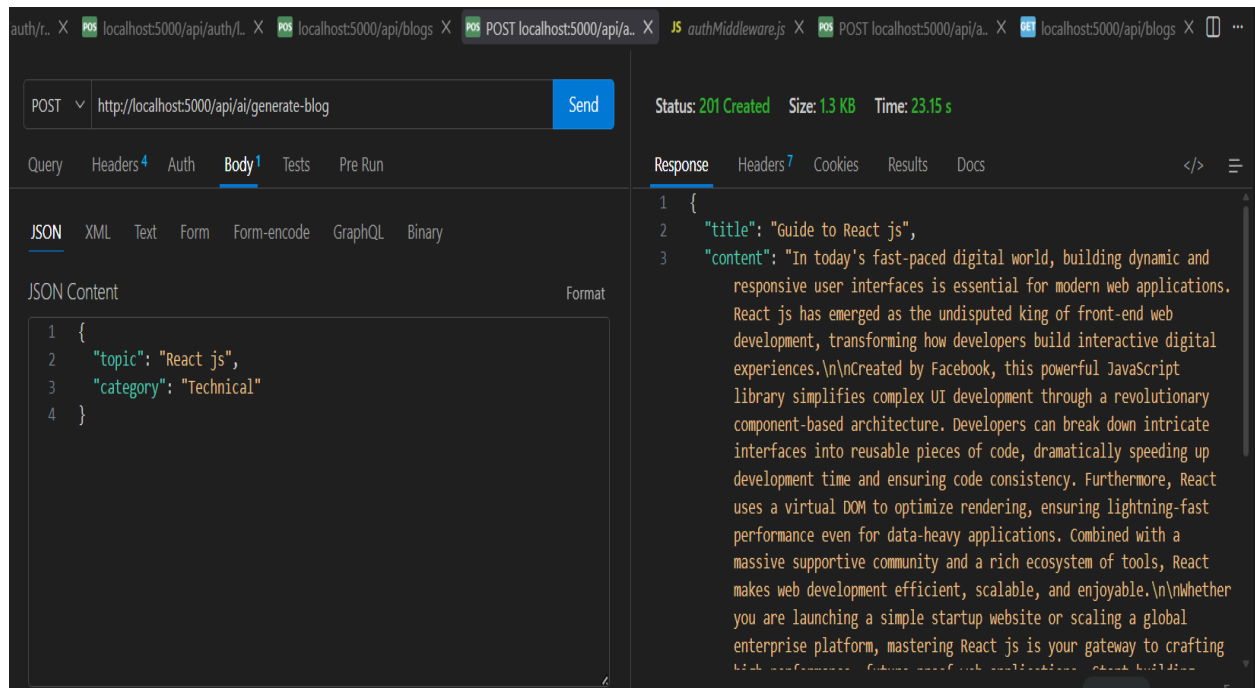
Create Blogs :



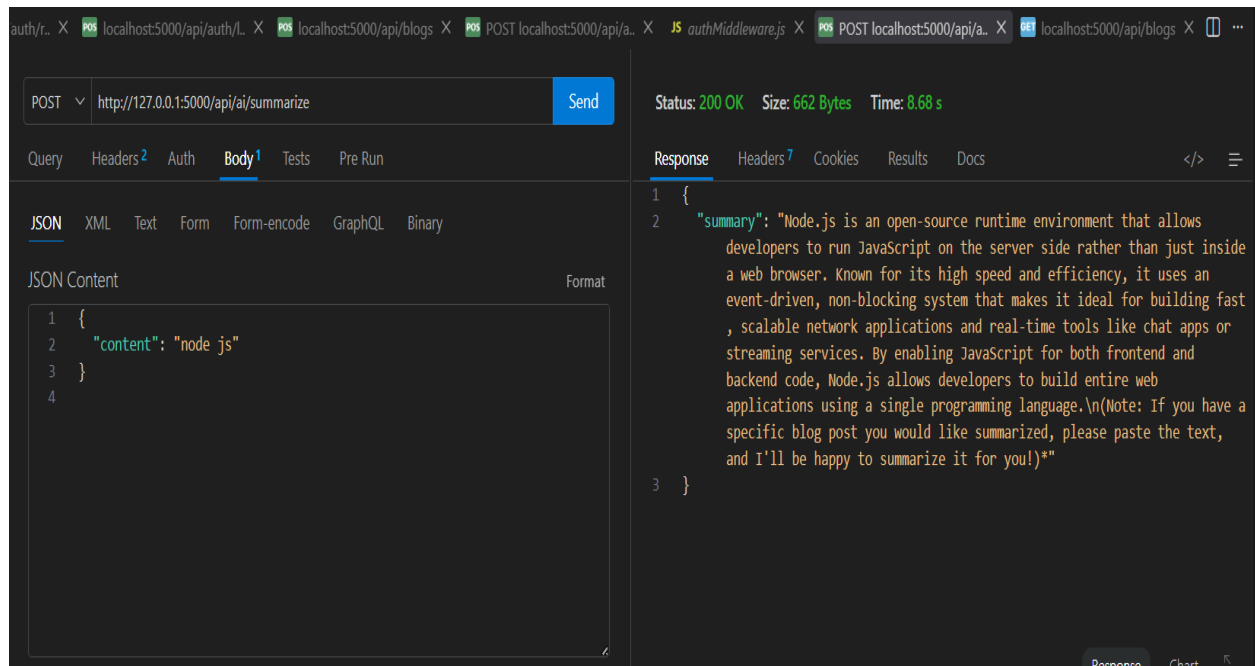
Fetch Blogs :



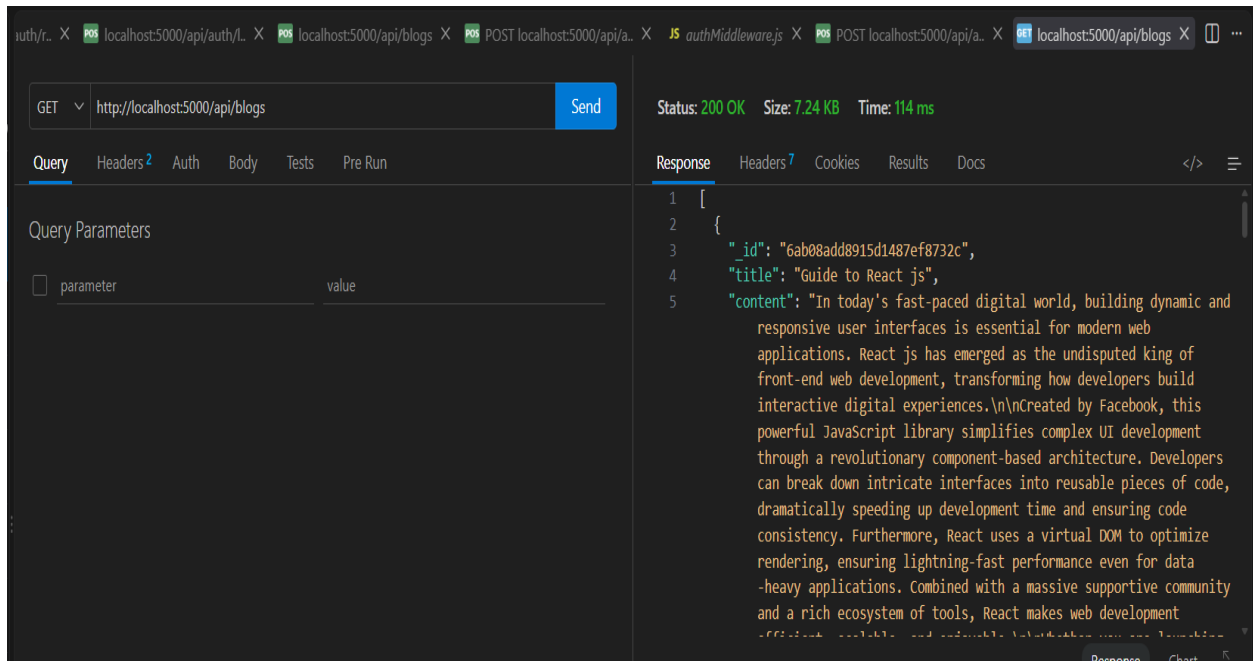
Generate AI blogs :



Summarize Blogs :



Get blogs:



Complete API Endpoint Table:

Method	Endpoint	Purpose	Authentication
POST	/api/auth/register	Register user	No
POST	/api/auth/login	Login user	No
POST	/api/blogs	Create blog	Yes
GET	/api/blogs/:id	Get one blog	No/Yes
POST	/api/ai/generate	Generate blog	Yes
POST	/api/ai/summarize	Summarize blog	Yes

Expected Outputs:

Registration

User registered successfully

Login

Login successful

JWT token generated

Blog Creation

Blog created successfully

Blog Retrieval

Blogs retrieved successfully

AI Generation

AI-generated blog content displayed

AI Summarization

Summary generated successfully

Advantages

1. Secure user authentication.
2. Easy blog management.
3. MongoDB provides flexible data storage.
4. REST APIs make the system scalable.
5. JWT provides token-based authentication.

Conclusion

AI BlogNest is an AI-powered blog management system that combines traditional blogging functionality with Artificial Intelligence.

The system provides essential features such as user registration, authentication, blog creation, blog management, and secure database storage. The integration of Gemini AI provides advanced features such as automatic blog generation and summarization.

The project demonstrates the practical implementation of:

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- REST APIs
- Google Gemini AI